

Product Management System

Add Product

Product List

No products found.

Update Product

Delete Product

Add Product

Product List

Refresh Products

ID: 2

Name: phone

Description: vivo

Price: ₹20000

Quantity: 5

ID: 3

Name: ASUS

Description: GAMING LAPTOP

Price: ₹60000

Quantity: 2

Update Product

2

POCO

Gaminig Phone

25000

2

Update Product

ID: 2
Name: POCO
Description: Gaminig Phone
Price: ₹25000
Quantity: 2

ID: 3
Name: ASUS
Description: GAMING LAPTOP
Price: ₹60000
Quantity: 2

Product List

Refresh Products

ID: 3
Name: ASUS
Description: GAMING LAPTOP
Price: ₹60000
Quantity: 2

Klh dbms - Spring Tool Suite 4

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- JRE System Library [JavaSE-17]
- Maven Dependencies
- src
- target
- HELP.md
- mvnw
- mvnw.cmd
- pom.xml
- week-9-CO3-2 [boot] [devtools] [Downloads main]
- src/main/java
- src/main/resources
- src/test/java
- JRE System Library [JavaSE-17]
- Maven Dependencies
- target
- HELP.md
- mvnw
- mvnw.cmd
- pom.xml

1 / pom.xml

```
1 spring.application.name=week-9-CO3-2
2 spring.datasource.url=jdbc:mysql://localhost:3306/week9_db
3 spring.datasource.username=root
4 spring.datasource.password=Charan123
5 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
6
7 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
8 spring.jpa.hibernate.ddl-auto=update
9 spring.jpa.show-sql=true
10
11 server.port=8080
```

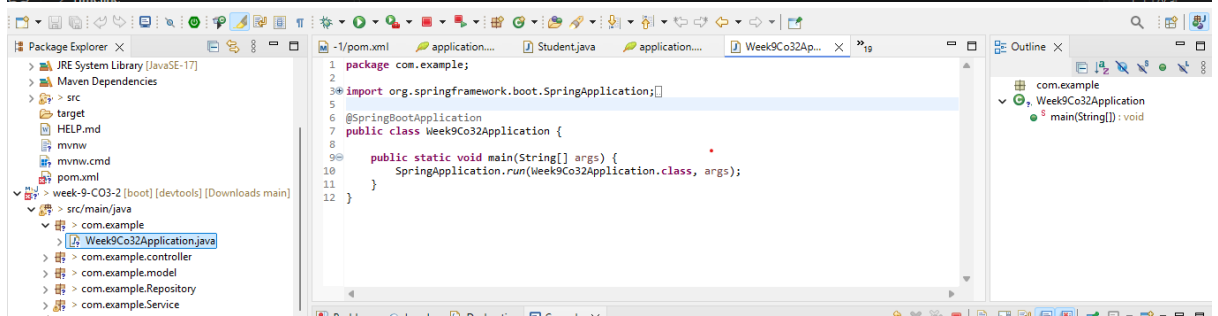
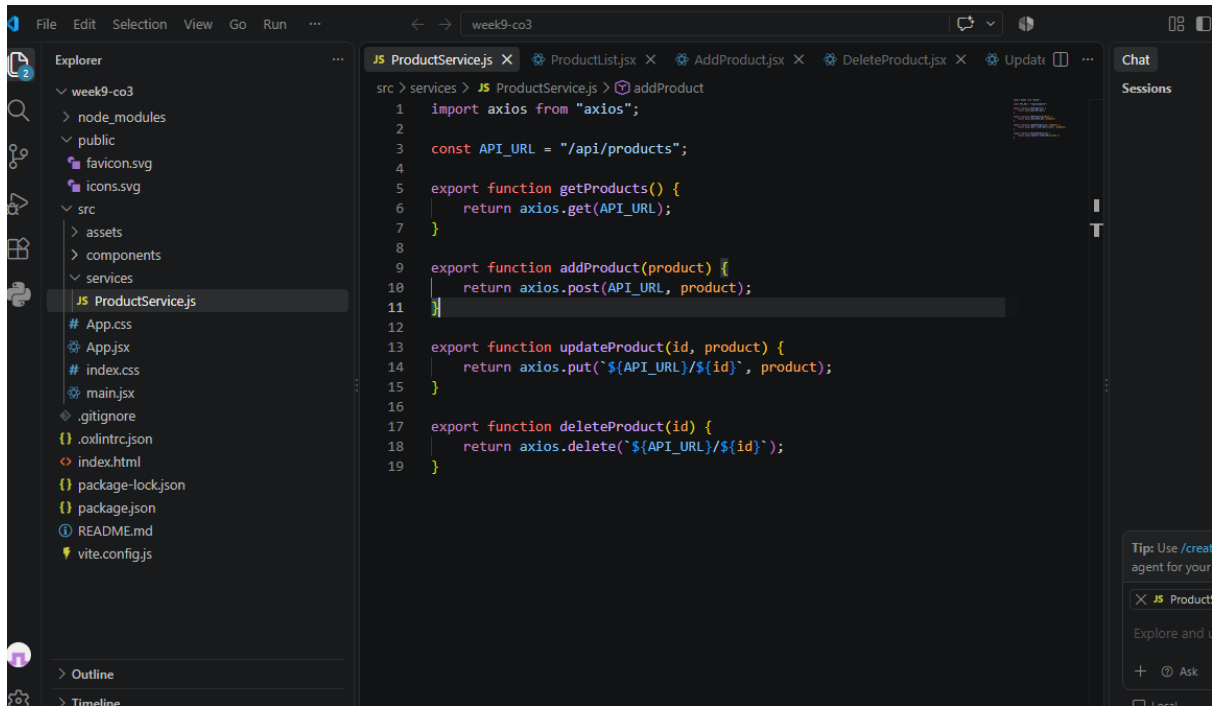
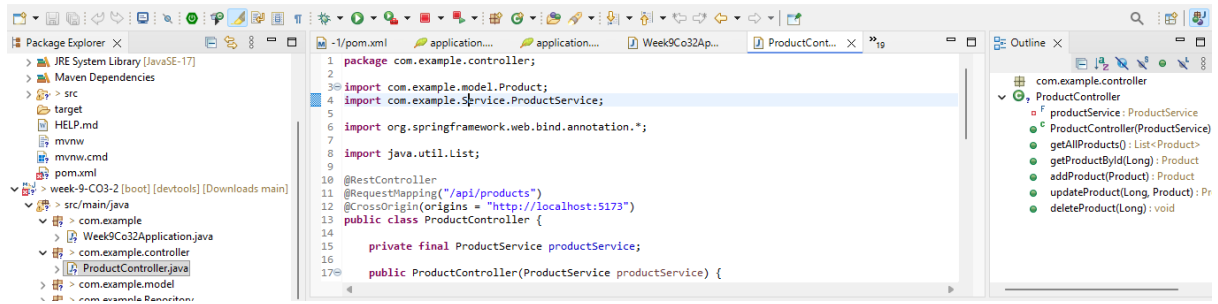
Problems @ Javadoc Declaration Console

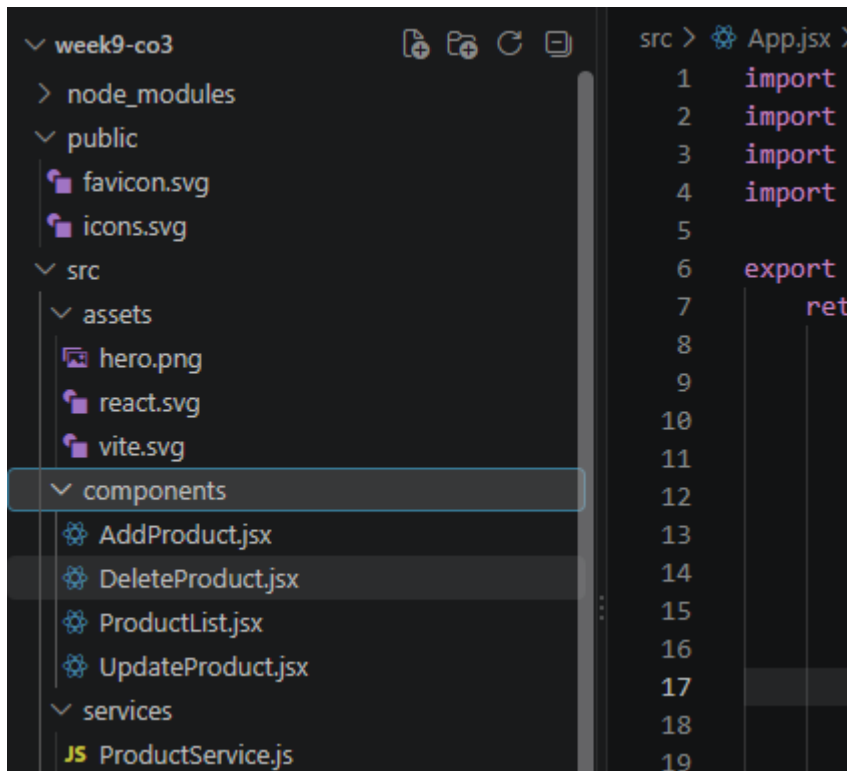
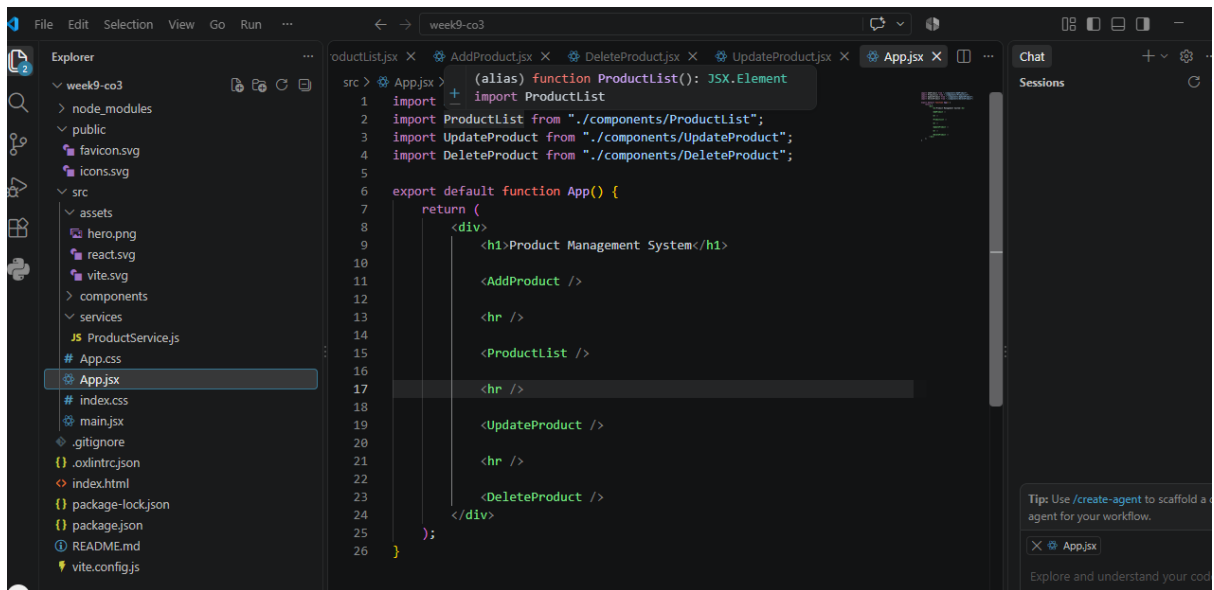
```
2026-09-19T08:31:14.007+05:30 INFO 11268 --- [week-9-CO3-2] [ restartedMain] com.example.Week9Co32Application : Started Week9Co32App
2026-09-19T08:31:25.600+05:30 INFO 11268 --- [week-9-CO3-2] [nio-8080-exec-1] o.s.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring D
2026-09-19T08:31:25.600+05:30 INFO 11268 --- [week-9-CO3-2] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet
2026-09-19T08:31:25.600+05:30 INFO 11268 --- [week-9-CO3-2] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initializat

Hibernate: insert into products (description,name,price,quantity) values (?,?,?,?)
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0
Hibernate: select count(*) from products p1_0 where p1_0.id=?
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0 where p1_0.id=?
Hibernate: update products set description=?,name=?,price=?,quantity=? where id=?
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0 where p1_0.id=?
Hibernate: delete from products where id=?
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0
```

3 elements hidden by filter

week-9-CO3-2





```
import { useState } from "react";
import { addProduct } from "../services/ProductService";

export default function AddProduct() {

  const [name, setName] = useState("");
  const [description, setDescription] = useState("");
  const [price, setPrice] = useState("");
  const [quantity, setQuantity] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();

    const product = {
      name: name,
      description: description,
      price: Number(price),
      quantity: Number(quantity)
    };

    try {
      await addProduct(product);

      alert("Product added successfully!");

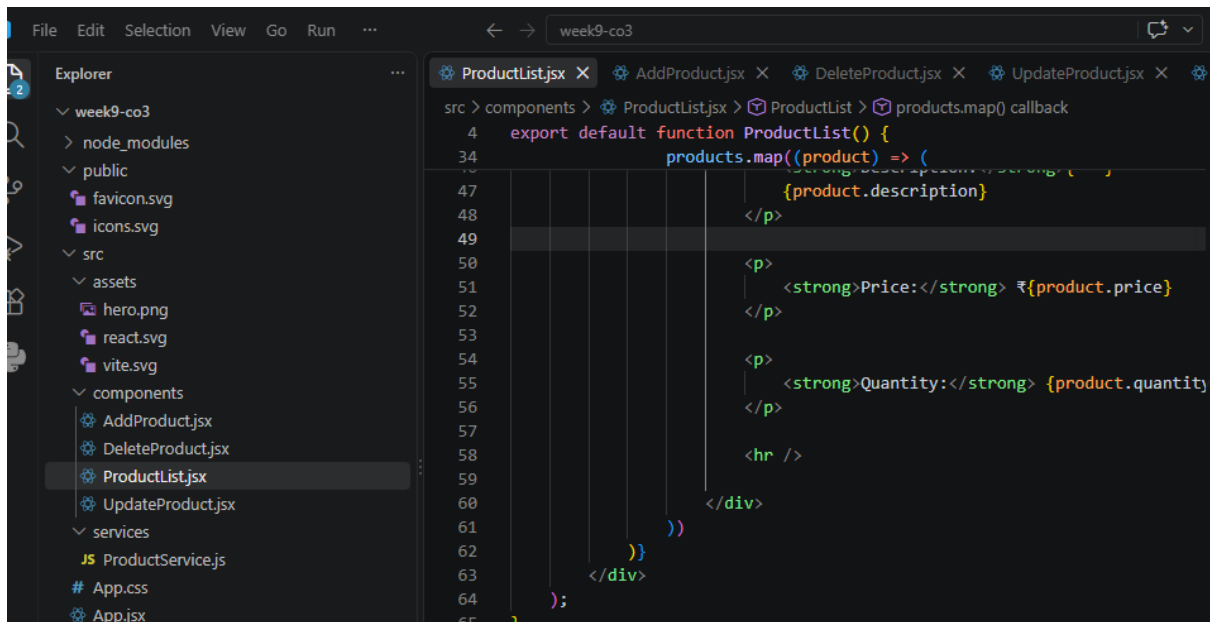
      setName("");
      setDescription("");
      setPrice("");
      setQuantity("");
    } catch (error) {
```

```
1 import { useState } from "react";
2 import { deleteProduct } from "../services/ProductService";
3
4 export default function DeleteProduct() {
5
6     const [id, setId] = useState("");
7
8     const handleSubmit = async (e) => {
9         e.preventDefault();
10
11         try {
12             await deleteProduct(id);
13
14             alert("Product deleted successfully!");
15             function alert(message?: any): void
16                 window.alert() instructs the browser to display a dialog with an optional message, and to wait until
17                 user dismisses the dialog.
18                 MDN Reference
19             alert("Failed to delete product");
20         } catch (error) {
21             console.error(error);
22         }
23     };
24
25     return (
26         <div>
27             <h2>Delete Product</h2>
28
29             <form onSubmit={handleSubmit}>
30
31                 <input
32                     type="number"
33                     placeholder="Product ID"
34                 />
35             </form>
36         </div>
37     );
38 }
```

```
4 export default function AddProduct() {
11     const handleSubmit = async (e) => {
12
13         try {
14             await addProduct(product);
15
16             alert("Product added successfully!");
17
18             setName("");
19             setDescription("");
20             setPrice("");
21             setQuantity("");
22         } catch (error) {
23             console.error(error);
24             alert("Failed to add product");
25         }
26     };
27
28     return (
29         <div>
30             <h2>Add Product</h2>
31
32             <form onSubmit={handleSubmit}>
33
34                 <input
35                     type="text"
36                     placeholder="Product Name"
37                     value={name}
38                     onChange={(e) => setName(e.target.value)}
39                     required
40                 />
41             </form>
42         </div>
43     );
44 }
```

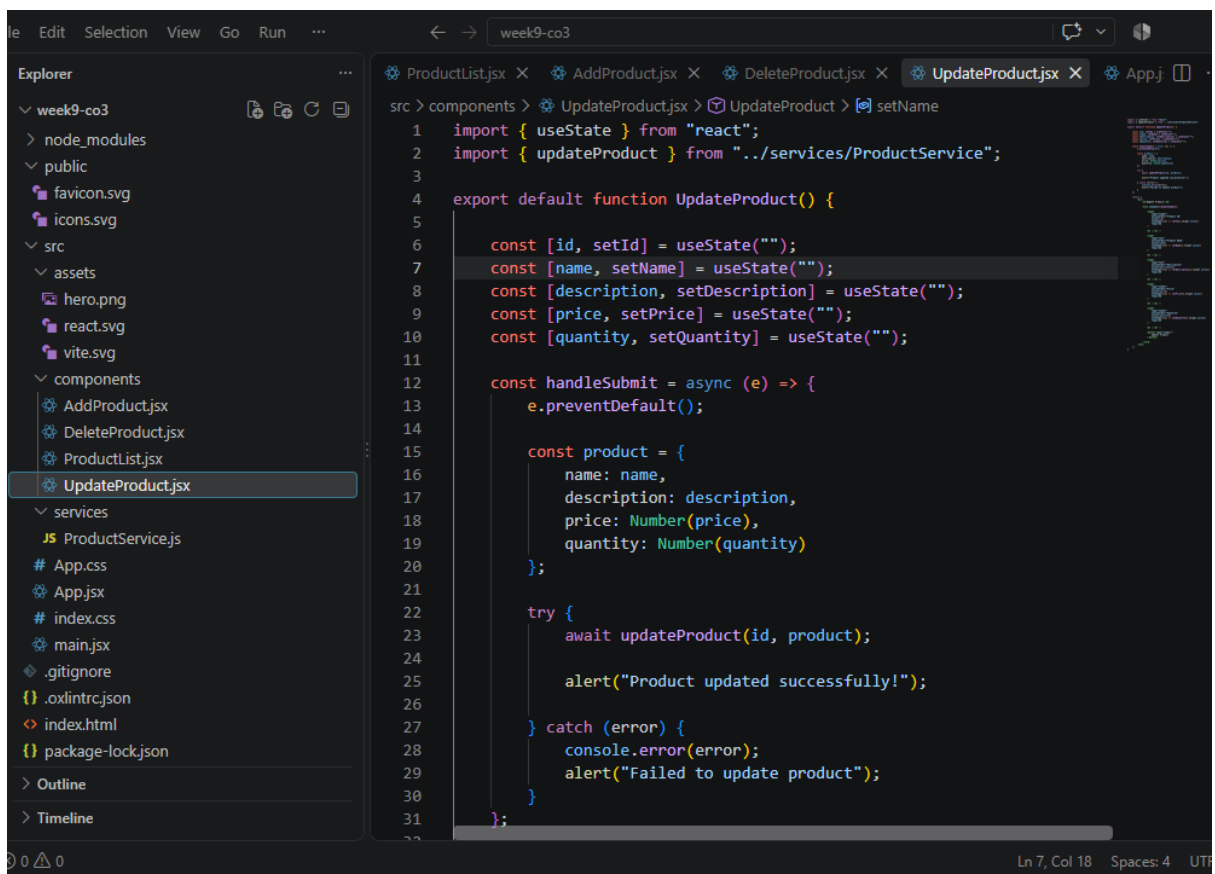
```
src > components > ProductList.jsx > ProductList > products.map() callback
1  import { useEffect, useState } from "react";
2  import { getProducts } from "../services/ProductService.js";
3
4  export default function ProductList() {
5
6      const [products, setProducts] = useState([]);
7
8      const loadProducts = async () => {
9          try {
10             const response = await getProducts();
11             setProducts(response.data);
12          } catch (error) {
13             console.error(error);
14          }
15      };
16
17      useEffect(() => {
18          loadProducts();
19      }, []);
20
21      return (
22          <div>
23              <h2>Product List</h2>
24
25              <button onClick={loadProducts}>
26                  Refresh Products
27              </button>
28
29              <br /><br />
30
31              {products.length === 0 ? (
32                  <p>No products found.</p>
33              ) : (
34                  products.map((product) => (
35                      <div key={product.id}>
36                          <p>
37                              <strong>ID:</strong> {product.id}
38                          </p>
39                          <p>
40                              <strong>Name:</strong> {product.name}
41                          </p>
42                          <p>
43                              <strong>Description:</strong>{" "}
44                              {product.description}
45                          </p>
46                      </div>
47                  )
48              )
49          </div>
50      );
51  }
```

```
src > components > ProductList.jsx > ProductList > products.map() callback
4  export default function ProductList() {
5
6      const [products, setProducts] = useState([]);
7
8      const loadProducts = async () => {
9          try {
10             const response = await getProducts();
11             setProducts(response.data);
12          } catch (error) {
13             console.error(error);
14          }
15      };
16
17      useEffect(() => {
18          loadProducts();
19      }, []);
20
21      return (
22          <div>
23              <h2>Product List</h2>
24
25              <button onClick={loadProducts}>
26                  Refresh Products
27              </button>
28
29              <br /><br />
30
31              {products.length === 0 ? (
32                  <p>No products found.</p>
33              ) : (
34                  products.map((product) => (
35                      <div key={product.id}>
36                          <p>
37                              <strong>ID:</strong> {product.id}
38                          </p>
39                          <p>
40                              <strong>Name:</strong> {product.name}
41                          </p>
42                          <p>
43                              <strong>Description:</strong>{" "}
44                              {product.description}
45                          </p>
46                      </div>
47                  )
48              )
49          </div>
50      );
51  }
```

This screenshot shows the VS Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for 'week9-co3', including 'node_modules', 'public', 'src', and 'services'. The 'src' folder is expanded, showing 'components', 'assets', and 'services'. The 'components' folder is selected, and 'ProductList.jsx' is highlighted. The code editor shows the 'ProductList.jsx' file, which is a React component that uses the 'products.map()' method to render a list of products. The code is as follows:

```
src > components > ProductList.jsx > ProductList > products.map() callback
4 export default function ProductList() {
34   products.map((product) => (
47     {product.description}
48   )
49 )
50   <p>
51     <strong>Price:</strong> ₹{product.price}
52   </p>
53   <p>
54     <strong>Quantity:</strong> {product.quantity}
55   </p>
56   <hr />
57 )
58 }
59 </div>
60 </div>
61 </div>
62 </div>
63 </div>
64 </div>
65 </div>
```



This screenshot shows the VS Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for 'week9-co3', including 'node_modules', 'public', 'src', and 'services'. The 'src' folder is expanded, showing 'components', 'assets', and 'services'. The 'components' folder is selected, and 'UpdateProduct.jsx' is highlighted. The code editor shows the 'UpdateProduct.jsx' file, which is a React component that uses the 'useState' hook to manage state and the 'updateProduct' function from the 'ProductService' to update a product. The code is as follows:

```
src > components > UpdateProduct.jsx > UpdateProduct > setName
1 import { useState } from "react";
2 import { updateProduct } from "../services/ProductService";
3
4 export default function UpdateProduct() {
5
6   const [id, setId] = useState("");
7   const [name, setName] = useState("");
8   const [description, setDescription] = useState("");
9   const [price, setPrice] = useState("");
10  const [quantity, setQuantity] = useState("");
11
12  const handleSubmit = async (e) => {
13    e.preventDefault();
14
15    const product = {
16      name: name,
17      description: description,
18      price: Number(price),
19      quantity: Number(quantity)
20    };
21
22    try {
23      await updateProduct(id, product);
24
25      alert("Product updated successfully!");
26
27    } catch (error) {
28      console.error(error);
29      alert("Failed to update product");
30    }
31  };
32 }
```

```
src > components > UpdateProduct.jsx > UpdateProduct > setName
4 export default function UpdateProduct() {
11
12     const handleSubmit = async (e) => {
13         e.preventDefault();
14
15         const product = {
16             name: name,
17             description: description,
18             price: Number(price),
19             quantity: Number(quantity)
20         };
21
22         try {
23             await updateProduct(id, product);
24
25             alert("Product updated successfully!");
26
27         } catch (error) {
28             console.error(error);
29             alert("Failed to update product");
30         }
31     };
32
33     return (
34         <div>
35             <h2>Update Product</h2>
36
37             <form onSubmit={handleSubmit}>
38
39                 <input
40                     type="number"
```

```
4 export default function UpdateProduct() {
31     };
32
33     return (
34         <div>
35             <h2>Update Product</h2>
36
37             <form onSubmit={handleSubmit}>
38
39                 <input
40                     type="number"
41                     placeholder="Product ID"
42                     value={id}
43                     onChange={(e) => setId(e.target.value)}
44                     required
45                 />
46
47                 <br /><br />
48
49                 <input
50                     type="text"
51                     placeholder="Product Name"
52                     value={name}
53                     onChange={(e) => setName(e.target.value)}
54                     required
55                 />
56
57                 <br /><br />
58
59                 <input
```

Charan Sai's Workspace

Search

InviteUpgrade

Spec Hub is your single home for API design and governance. Author specs visually, convert them to collections that stay in sync, and enforce standards across your team with custom governance rules.

Postman

> .postman

> postman

Overview

Overview

Files

GET http://localhost:8080/api/products

No environment

AI

GET

http://localhost:8080/api/products

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "name": "Gaming Laptop",
3   "description": "ASUS Gaming Laptop",
4   "price": 75000,
5   "quantity": 3
6 }
```

Body

Cookies

Headers 6

Test Results

200 OK

52 ms

316 B

{}

JSON

Preview

Visualize

```
1 [
2   {
3     "description": "GAMING LAPTOP",
4     "id": 3,
5     "name": "ASUS",
6     "price": 60000.0,
7     "quantity": 2
8   }
9 ]
```

New Chat

Describe what you context, / for Skills

Auto